HDL

1. Introduction

1.1 Purpose

- This document describes the high level design of Logistics Management System.
- It describes the overall architecture, major components, and interactions of the system.
- It translates the SRS requirements into a design structure.

1.2 Scope

- The system will include the logistics portal, web interfaces, shipment booking, tracking, route management and delivery verification
- The system will use some external systems like Google maps for route management and payment gateways for payment processing.

2. Architectural Analysis

2.1 Key Functional Requirements

- Customer can book shipment
- Customer can track order status
- Driver receives delivery route
- Driver confirms delivery by OTP
- Manager manages routes and orders
- Support staff interact with customers

2.2 Key Non Functional Requirements

- System should load parcel details under 2 seconds
- System must support 10,000 concurrent users
- Passwords must be hashed using bcrypt
- Communication must use HTTPS
- System must be available during business hours

2.3 Constraints

- PostgreSQL database
- Linux server environment
- Google Maps Apl
- Third party payment gateway

2.4 Architectural Drivers

- Multiple user roles
- Real time shipment tracking
- Scalability for many users
- Integration with external APIs
- Secure authentication

3. Architecture Overview

3.1 Selected Architectural Style

- The suitable architecture is Layered architecture. As mobile/web interfaces act as clients and they can share the same backend logic and database

3.2 Deployment Assumptions

- The system runs in browser
- Backend service runs on linux server
- Database hosted on PostgreSQL server

3.3 High -Level Architecture Description

- There would be a client layer which interacts with the user and send the data to API layer.
- API layer processes the data from the Client layer and triggers the correct business logic by sending it to Business Logic Layer.
- If need Business Logic layer communicates with the Data Layer for data access.

4. High-Level System Decomposition

4.1 Frontend Components

- Customer Web portal
- Driver Web portal
- Manager web portal
- Support Staff portal

4.2 Backend Services

- Authentication Service
- Order Management Service
- Tracking Service
- Route Management Service
- Delivery Verification Service
- Support Chat Service

4.3 External Systems

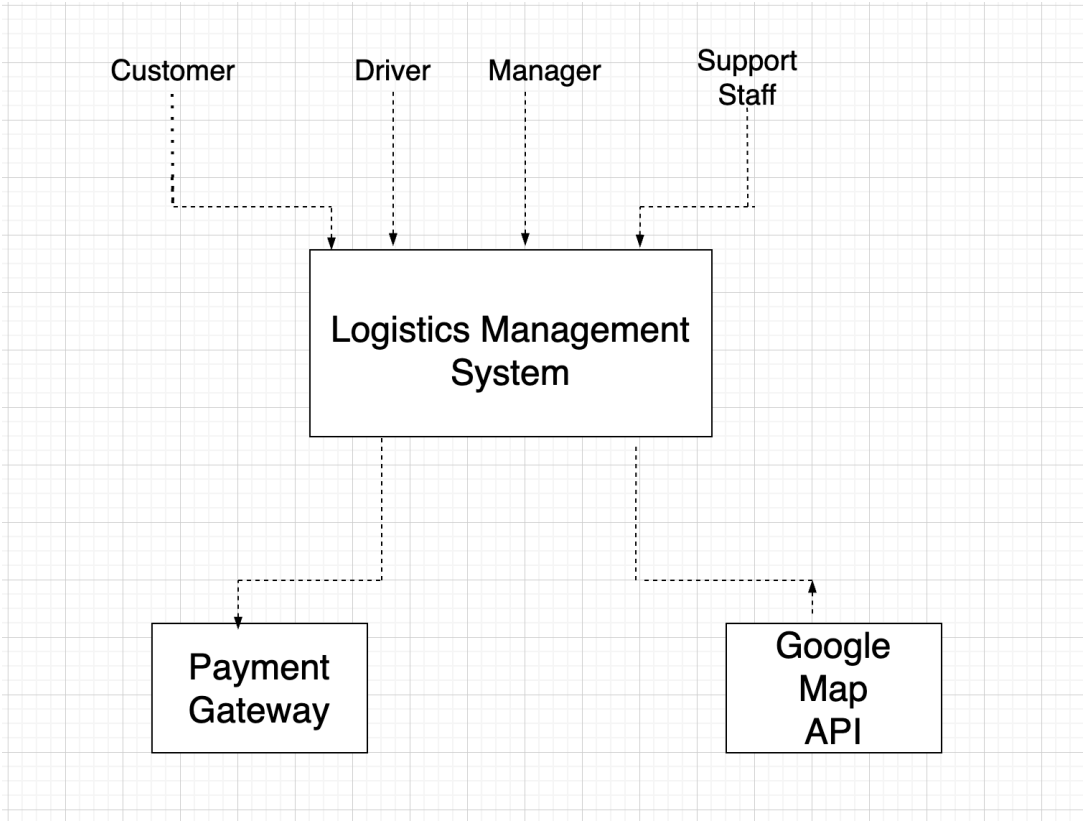
- Google Maps API for routing
- Payment Gateways for payment processing

4.4 Data Storage

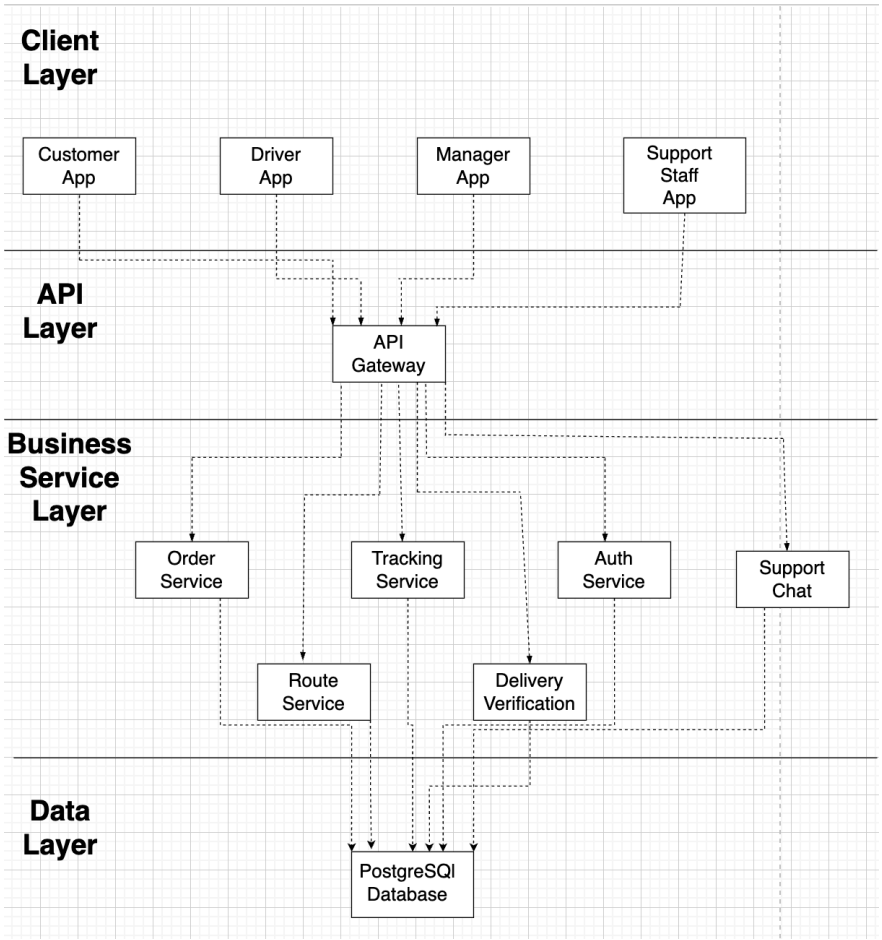
- PostgreSQL database storing:
    - User accounts
    - Shipment records
    - Order status
    - Route data

5. UML Design Diagrams

5.1 System Context Diagram

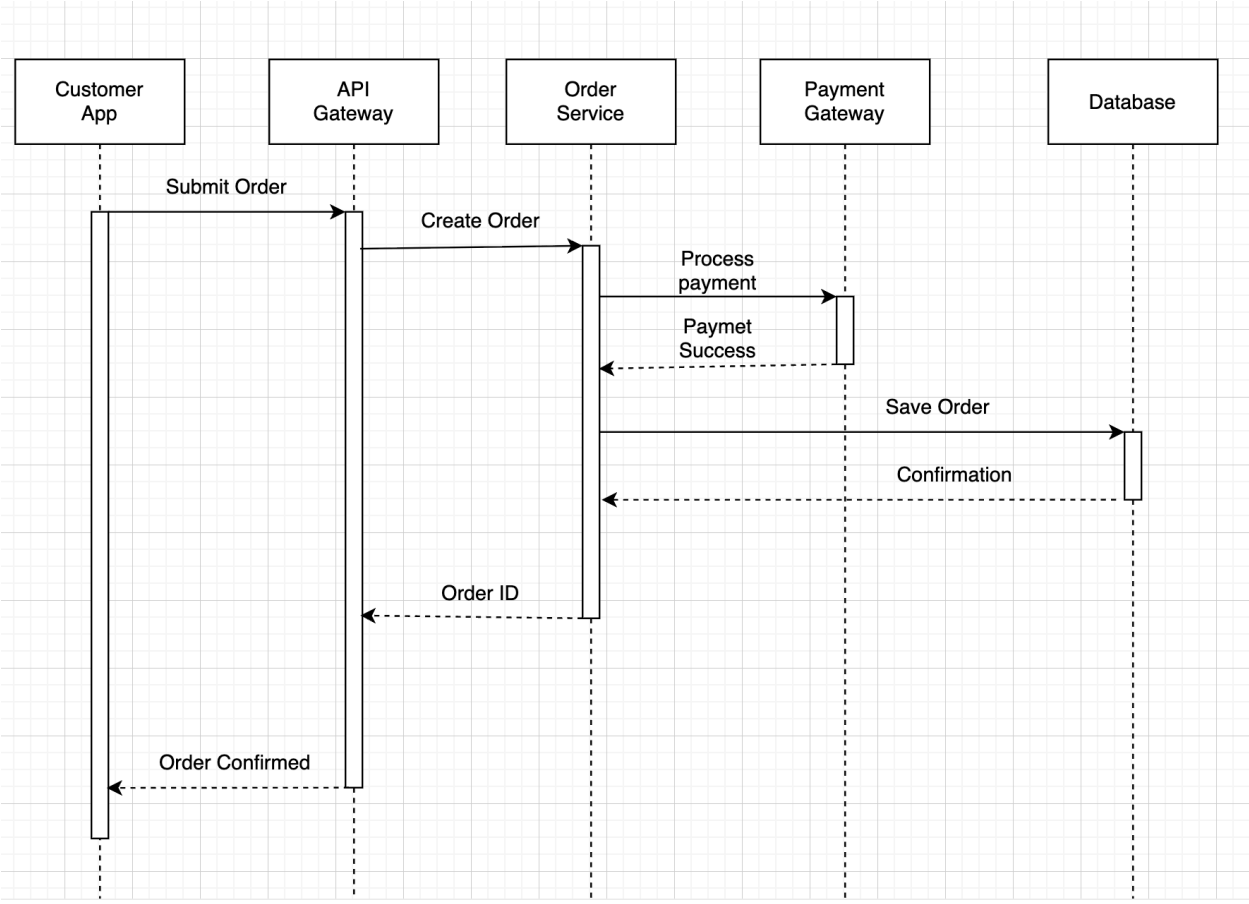


5.2 High Level Component Design



5.3 Sequence Diagram

Book Shipment



6. Traceability Matrix

SRS Requirement	Design Component
FR-C-01	Order Management Service
FR-C-02	Tracking Service
FR-D-03	Delivery Verification
FR-M-03	Route Service

7. Design Rationale

- A layered architecture is used to separate presentation, business logic, and data layers.
- Each major functionality is implemented as a separate service, this reduces dependency between components.
- Backend services can be scaled horizontally.
- API Gateway manages high number of requests.

8. Assumptions and Risks

8.1 Assumptions

- Drivers have basic knowledge of English to operate the application.
- Continuous internet connectivity is available.
- Users have access to mobile devices with internet.

8.2 Risks

- Changes in Google Maps API may affect routing functionality.
- Network connectivity issues may disrupt real-time tracking
- Payment gateway failures may interrupt order booking..